

# Symbolic Analysis

## EdgeBasedModels.jl Vignette 07

Simon Frost

2026-03-27

### Table of contents

|                                                       |    |
|-------------------------------------------------------|----|
| Introduction . . . . .                                | 1  |
| Setup . . . . .                                       | 2  |
| Symbolic $R_0$ for SIR . . . . .                      | 2  |
| Numeric $R_0$ Evaluation . . . . .                    | 3  |
| $R_0$ for SEIR . . . . .                              | 3  |
| Multi-stage $R_0$ . . . . .                           | 4  |
| Inspecting Model Equations . . . . .                  | 5  |
| Compact form . . . . .                                | 6  |
| Expanded form . . . . .                               | 6  |
| SEIR equations . . . . .                              | 7  |
| Parameter Sensitivity of $R_0$ . . . . .              | 8  |
| $R_0$ versus mean degree $\kappa$ . . . . .           | 8  |
| $R_0$ versus transmission rate $\beta$ . . . . .      | 9  |
| Multi-stage $R_0$ sensitivity . . . . .               | 10 |
| PGF Analysis . . . . .                                | 11 |
| Poisson PGF derivatives . . . . .                     | 11 |
| Polynomial PGF . . . . .                              | 12 |
| Comparing $R_0$ across degree distributions . . . . . | 12 |
| Summary . . . . .                                     | 14 |

### Introduction

One of the key strengths of EdgeBasedModels.jl is that all model equations are generated symbolically using [Symbolics.jl](#) and [ModelingToolkit.jl](#). This means we can inspect, simplify, and analyse model equations before running any simulation.

This vignette demonstrates:

- Computing  $R_0$  symbolically for SIR, SEIR, and multi-stage models
- Evaluating symbolic expressions numerically
- Inspecting the ODE equations generated by model builders
- Sensitivity analysis of  $R_0$  to model parameters
- Working with probability generating functions (PGFs) symbolically

## Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using Plots
```

## Symbolic $R_0$ for SIR

The basic reproduction number  $R_0$  for an edge-based model on a configuration-model network is:

$$R_0 = T \cdot \frac{\psi''(1)}{\psi'(1)}$$

where  $T$  is the transmissibility (probability of transmission across a single edge before recovery) and  $\psi''(1)/\psi'(1)$  is the excess degree ratio — the expected number of additional edges a neighbour has beyond the one connecting it to the focal node.

For a single-stage SIR with transmission rate  $\beta$  and recovery rate  $\gamma$ , the transmissibility is  $T = \beta/(\beta + \gamma)$ . For a Poisson network with mean degree  $\kappa$ , the excess degree ratio equals  $\kappa$ , giving:

$$R_0 = \frac{\beta\kappa}{\beta + \gamma}$$

Let us compute this symbolically with `basic_reproduction_number`:

```
@parameters β γ κ
pgf = poisson_pgf(κ)

sir_prog = DiseaseProgression(
    [DiseaseStage(:I; transmission_rate=β), DiseaseStage(:R)],
    [DiseaseTransition(:I, :R, γ)];
    entry=:I
```

```
)
scm = StaticConfigurationModel(pgf, sir_prog)
R0_sir = basic_reproduction_number(scm); nothing
```

```
println("Symbolic  $R_0$  (SIR): ", R0_sir)
```

Symbolic  $R_0$  (SIR):  $(\beta \kappa) / (\beta + \gamma)$

The result is a symbolic expression in terms of  $\beta$ ,  $\gamma$ , and  $\kappa$  that can be manipulated, simplified, or evaluated numerically.

## Numeric $R_0$ Evaluation

We evaluate  $R_0$  at specific parameter values by substituting into the symbolic expression:

```
R0_val = Symbolics.value(
    Symbolics.substitute(R0_sir, Dict{β ⇒ 0.6, γ ⇒ 0.1, κ ⇒ 5.0})
)
println("R0 = ", R0_val)
println("Expected:  $\beta\kappa/(\beta+\gamma)$  = ", 0.6 * 5.0 / (0.6 + 0.1))
```

$R_0 = 4.285714285714286$   
 Expected:  $\beta\kappa/(\beta+\gamma) = 4.285714285714286$

With  $\beta = 0.6$ ,  $\gamma = 0.1$ , and  $\kappa = 5$ , we get  $R_0 = 3/0.7 \approx 4.29$ , confirming the analytic formula.

## $R_0$ for SEIR

Adding a non-infectious exposed stage  $E$  does not change  $R_0$ , because  $E$  does not contribute to transmission. The transmissibility  $T$  depends only on the infectious stages.

```
@parameters σ

seir_prog = DiseaseProgression(
    [
        DiseaseStage(:E; transmission_rate=0),
        DiseaseStage(:I; transmission_rate=β),
        DiseaseStage(:R)
```

```

],
[
    DiseaseTransition(:E, :I, σ),
    DiseaseTransition(:I, :R, γ)
];
entry=:E
)
scm_seir = StaticConfigurationModel(pgf, seir_prog)
R0_seir = basic_reproduction_number(scm_seir)
println("Symbolic R0 (SEIR): ", R0_seir)

```

Symbolic  $R_0$  (SEIR):  $(\beta \cdot \kappa) / (\beta + \gamma)$

```

R0_seir_val = Symbolics.value(
    Symbolics.substitute(R0_seir, Dict{β ⇒ 0.6, γ ⇒ 0.1, σ ⇒ 0.2, κ ⇒ 5.0})
)
println("SEIR R0 = ", R0_seir_val)
println("SIR R0 = ", R0_val)
println("Equal? ", R0_seir_val ≈ R0_val)

```

```

SEIR R0 = 4.285714285714286
SIR R0 = 4.285714285714286
Equal? true

```

As expected, the SEIR  $R_0$  equals the SIR  $R_0$  because the latent period affects the epidemic *timing* but not the *threshold*.

## Multi-stage $R_0$

For diseases with multiple infectious stages (e.g., acute and chronic phases with different transmission rates), the transmissibility becomes:

$$T = 1 - \prod_{m \in \text{infectious}} \frac{\gamma_m}{\beta_m + \gamma_m}$$

where the product is over all infectious stages. Each factor  $\gamma_m/(\beta_m + \gamma_m)$  is the probability of progressing *without* transmitting in stage  $m$ , so  $T$  is the probability of transmitting in at least one stage.

```

@parameters β1 β2 γ1 γ2

twostage_prog = DiseaseProgression(
    [
        DiseaseStage(:I1; transmission_rate=β1),
        DiseaseStage(:I2; transmission_rate=β2),
        DiseaseStage(:R)
    ],
    [
        DiseaseTransition(:I1, :I2, γ1),
        DiseaseTransition(:I2, :R, γ2)
    ];
    entry=:I1
)
scm_twostage = StaticConfigurationModel(pgf, twostage_prog)
R0_twostage = basic_reproduction_number(scm_twostage)
println("Symbolic R0 (two-stage): ", R0_twostage)

```

Symbolic  $R_0$  (two-stage):  $(((\beta_1 + \gamma_1) * (\beta_2 + \gamma_2) - \gamma_1 * \gamma_2) * \kappa) / ((\beta_1 + \gamma_1) * (\beta_2 + \gamma_2))$

```

R0_twostage_val = Symbolics.value(
    Symbolics.substitute(R0_twostage,
        Dict{β1 ⇒ 0.3, γ1 ⇒ 0.2, β2 ⇒ 0.1, γ2 ⇒ 0.3, κ ⇒ 5.0})
)
expected_T = 1 - (0.2 / (0.3 + 0.2)) * (0.3 / (0.1 + 0.3))
expected_R0 = expected_T * 5.0
println("Computed R0 = ", R0_twostage_val)
println("Expected T = ", expected_T)
println("Expected R0 = ", expected_R0)
println("Match? ", R0_twostage_val ≈ expected_R0)

```

```

Computed R0 = 3.5
Expected T = 0.7
Expected R0 = 3.5
Match? true

```

## Inspecting Model Equations

EdgeBasedModels.jl generates ODE systems using ModelingToolkit.jl. We can inspect these equations symbolically before solving.

## Compact form

The compact (Miller) formulation uses just 2 ODEs for SIR:

```
compact_model = build_sir(pgf, β, γ; form=:compact)
compact_eqs = ModelingToolkit.equations(compact_model.system)
println("Compact SIR: ", length(compact_eqs), " ODEs")
for eq in compact_eqs
    println(" ", eq)
end
```

Compact SIR: 2 ODEs

$$\begin{aligned} \text{Differential}(t, 1)(R(t)) &\sim (1 - \exp((-1 + \theta(t))\kappa) - R(t))\gamma \\ \text{Differential}(t, 1)(\theta(t)) &\sim (-\exp((-1 + \theta(t))\kappa)\beta + \theta(t)\exp(0)\beta - (1 - \theta(t))\exp(0)\gamma) / (-\exp(0)) \end{aligned}$$

## Expanded form

The expanded formulation introduces explicit  $\varphi$  variables for each edge state:

```
expanded_model = build_sir(pgf, β, γ; form=:expanded)
expanded_eqs = ModelingToolkit.equations(expanded_model.system)
println("Expanded SIR: ", length(expanded_eqs), " ODEs")
for eq in expanded_eqs
    println(" ", eq)
end
```

Expanded SIR: 4 ODEs

$$\begin{aligned} \text{Differential}(t, 1)(R(t)) &\sim (1 - \exp((-1 + \theta(t))\kappa) - R(t))\gamma \\ \text{Differential}(t, 1)(\phi_R(t)) &\sim \phi_I(t)\gamma \\ \text{Differential}(t, 1)(\phi_I(t)) &\sim -\phi_I(t)(\beta + \gamma) + \phi_S(t)\phi_I(t)\beta\kappa \\ \text{Differential}(t, 1)(\theta(t)) &\sim -\phi_I(t)\beta \end{aligned}$$

We can also inspect the state variables and observables:

```
println("\nState variables (unknowns):")
for u in ModelingToolkit.unknowns(expanded_model.system)
    println(" ", u)
end
```

State variables (unknowns):

$R(t)$   
 $\phi_R(t)$   
 $\phi_I(t)$   
 $\theta(t)$

```
println("Observed (algebraic) variables:")
for obs in ModelingToolkit.observables(expanded_model.system)
    println(" ", obs)
end
```

Observed (algebraic) variables:

$\phi_S(t) \sim \exp((-1 + \theta(t))\kappa) / \exp(\theta)$   
 $S(t) \sim \exp((-1 + \theta(t))\kappa)$   
 $I(t) \sim 1 - R(t) - S(t)$

The expanded form makes the edge-level dynamics explicit: each  $\phi$  variable tracks the probability that a random test edge connects to a neighbour in a given disease stage. The compact form combines these into a single  $\theta$  equation using the PGF.

## SEIR equations

For more complex diseases, the number of equations grows with the number of disease stages:

```
seir_model = build_seir(pgf,  $\sigma$ ,  $\beta$ ,  $\gamma$ )
seir_eqs = ModelingToolkit.equations(seir_model.system)
println("Expanded SEIR: ", length(seir_eqs), " ODEs")
for eq in seir_eqs
    println(" ", eq)
end
```

Expanded SEIR: 5 ODEs

$\text{Differential}(t, 1)(R(t)) \sim (1 - \exp((-1 + \theta(t))\kappa) - R(t))\gamma$   
 $\text{Differential}(t, 1)(\phi_R(t)) \sim \phi_I(t)\gamma$   
 $\text{Differential}(t, 1)(\phi_I(t)) \sim \phi_E(t)\sigma - \phi_I(t)(\beta + \gamma)$   
 $\text{Differential}(t, 1)(\phi_E(t)) \sim -\phi_E(t)\sigma + \phi_S(t)\phi_I(t)\beta\kappa$   
 $\text{Differential}(t, 1)(\theta(t)) \sim -\phi_I(t)\beta$

## Parameter Sensitivity of $R_0$

Because  $R_0$  is a symbolic expression, we can evaluate it over parameter ranges to study sensitivity. This is much faster than sweeping parameters through ODE solutions.

$R_0$  versus mean degree  $\kappa$

```
κ_range = 0.1:0.1:10.0
R0_vs_κ = [
    Symbolics.value(Symbolics.substitute(R0_sir, Dict(β ⇒ 0.6, γ ⇒ 0.1, κ ⇒ κ_v)))
    for κ_v in κ_range
]

plot(κ_range, R0_vs_κ, lw=2, color=:steelblue, label="R0(κ)")
hline!([1.0], linestyle=:dash, color=:red, label="Epidemic threshold")
xlabel!("Mean degree κ")
ylabel!("R0")
title!("R0 vs Network Connectivity")
```

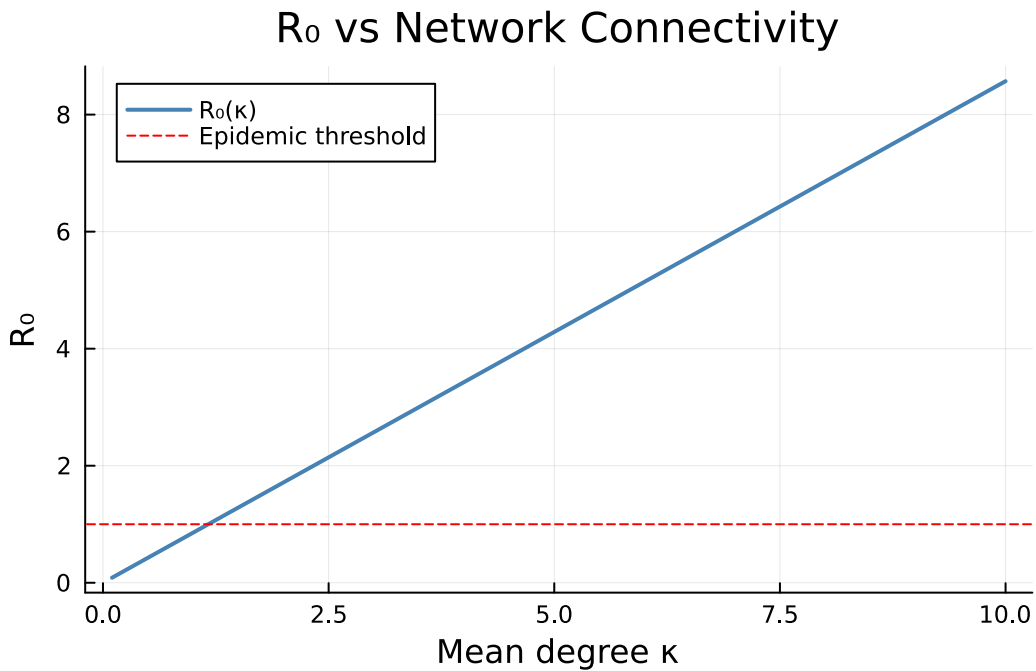


Figure 1:  $R_0$  as a function of mean network degree  $\kappa$ , for fixed  $\beta = 0.6$  and  $\gamma = 0.1$ . The dashed line marks the epidemic threshold  $R_0 = 1$ .



For a Poisson network with  $R_0 = \beta\kappa/(\beta+\gamma)$ , the epidemic threshold  $R_0 = 1$  occurs at  $\kappa^* = (\beta+\gamma)/\beta$ . With  $\beta = 0.6$  and  $\gamma = 0.1$ , this gives  $\kappa^* = 0.7/0.6 \approx 1.17$ .

$R_0$  versus transmission rate  $\beta$

```

β_range = 0.01:0.01:1.0
R0_vs_β = [
    Symbolics.value(Symbolics.substitute(R0_sir, Dict{β ⇒ β_v, γ ⇒ 0.1, κ ⇒ 5.0}))
    for β_v in β_range
]

plot(β_range, R0_vs_β, lw=2, color=:darkorange, label="R₀(β)")
hline!([1.0], linestyle=:dash, color=:red, label="Epidemic threshold")
xlabel!("Transmission rate β")
ylabel!("R₀")
title!("R₀ vs Transmission Rate")

```

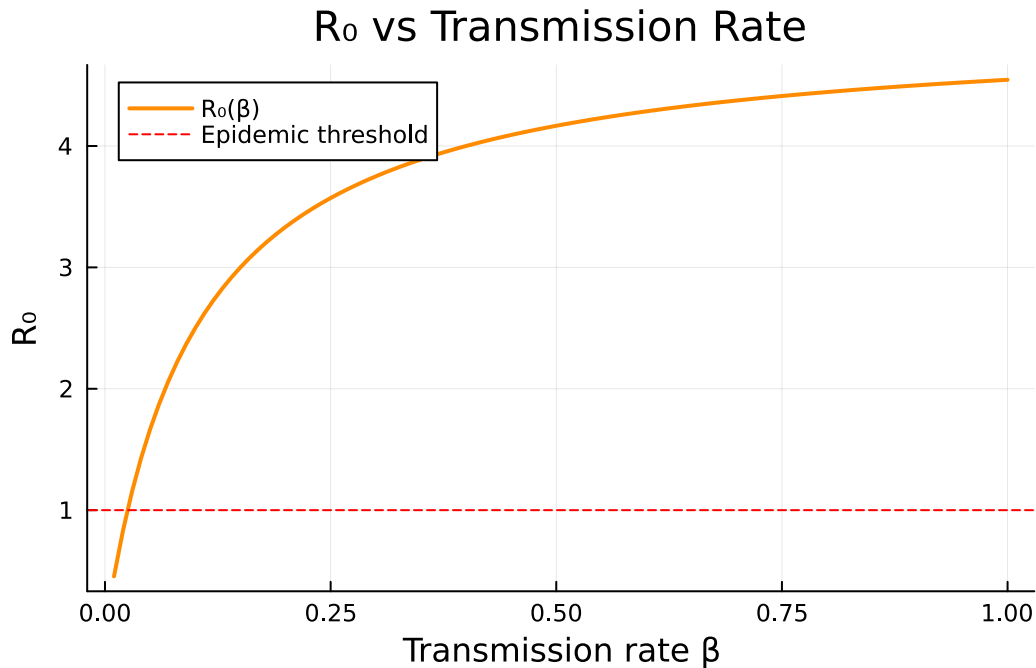


Figure 2:  $R_0$  as a function of transmission rate  $\beta$ , for fixed  $\kappa = 5$  and  $\gamma = 0.1$ .

$R_0$  increases with  $\beta$  but saturates at  $\kappa = 5$  as  $\beta \rightarrow \infty$  (since  $\beta/(\beta + \gamma) \rightarrow 1$ ).

## Multi-stage $R_0$ sensitivity

We can also examine how  $R_0$  depends on the relative transmission rates in a two-stage model:

```
 $\beta_1$ _range = 0.01:0.02:0.5
 $\beta_2$ _range = 0.01:0.02:0.5
R0_matrix = [
    Symbolics.value(Symbolics.substitute(R0_twostage,
        Dict( $\beta_1 \Rightarrow$  b1,  $\beta_2 \Rightarrow$  b2,  $\gamma_1 \Rightarrow$  0.2,  $\gamma_2 \Rightarrow$  0.3,  $\kappa \Rightarrow$  5.0)))
    for b1 in  $\beta_1$ _range, b2 in  $\beta_2$ _range
]

heatmap(collect( $\beta_1$ _range), collect( $\beta_2$ _range), R0_matrix',
    xlabel=" $\beta_1$  (stage 1 transmission)",
    ylabel=" $\beta_2$  (stage 2 transmission)",
    title="Two-stage  $R_0$ ",
    colorbar_title=" $R_0$ ",
    color=:viridis)
contour!(collect( $\beta_1$ _range), collect( $\beta_2$ _range), R0_matrix',
    levels=[1.0], linewidth=2, linecolor=:white, label="")
```

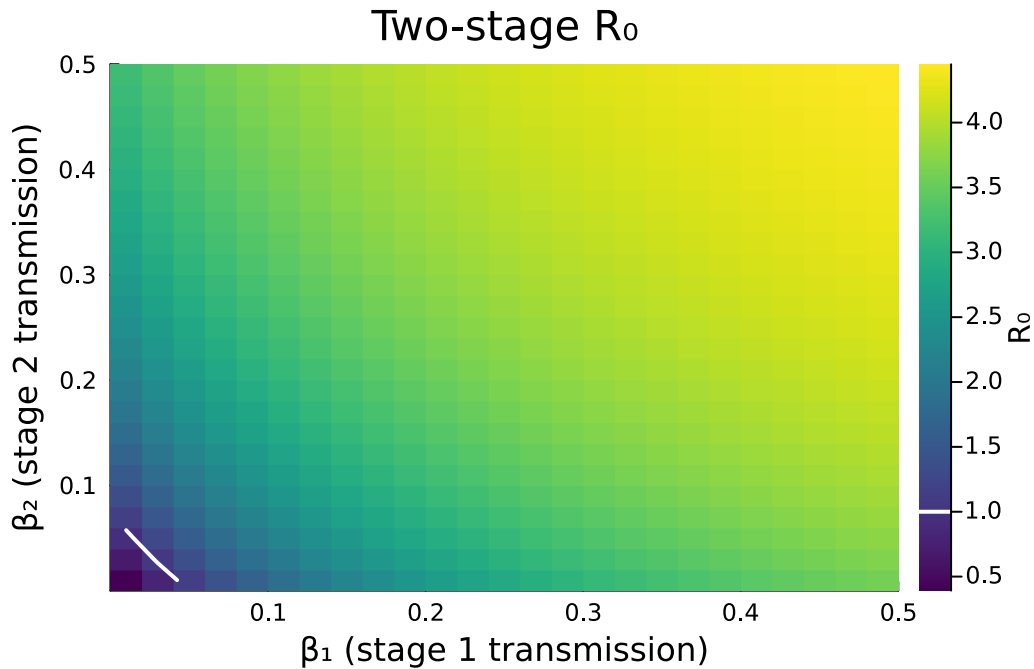


Figure 3: Two-stage  $R_0$  as a function of  $\beta_1$  and  $\beta_2$ , with  $\gamma_1 = 0.2$ ,  $\gamma_2 = 0.3$ ,  $\kappa = 5$ . The white contour marks  $R_0 = 1$ .

## PGF Analysis

The probability generating function (PGF) encodes the network's degree distribution and plays a central role in edge-based models. `EdgeBasedModels.jl` provides tools for symbolic manipulation of PGFs.

### Poisson PGF derivatives

For a Poisson network with mean degree  $\kappa$ , the PGF is  $\psi(z) = e^{\kappa(z-1)}$ :

```
pgf_poisson = poisson_pgf( $\kappa$ )
println("ψ(z) = ", pgf_poisson.expression)
```

$$\psi(z) = \exp((-1 + z) * \kappa)$$

The first and second derivatives are:

```
d1 = pgf_derivative(pgf_poisson, 1)
println("ψ'(z) = ", d1)

d2 = pgf_derivative(pgf_poisson, 2)
println("ψ''(z) = ", d2)
```

$$\begin{aligned}\psi'(z) &= \exp((-1 + z) * \kappa) * \kappa \\ \psi''(z) &= \exp((-1 + z) * \kappa) * (\kappa^2)\end{aligned}$$

The mean degree is  $\psi'(1)$  and the excess degree ratio is  $\psi''(1)/\psi'(1)$ :

```
mean_deg = mean_degree(pgf_poisson)
println("Mean degree ψ'(1) = ", mean_deg)

excess = Symbolics.simplify(
    Symbolics.substitute(d2, Dict{pgf_poisson.variable => 1}) /
    Symbolics.substitute(d1, Dict{pgf_poisson.variable => 1})
)
println("Excess degree ψ''(1)/ψ'(1) = ", excess)
```

$$\begin{aligned}\text{Mean degree } \psi'(1) &= \exp(0) * \kappa \\ \text{Excess degree } \psi''(1)/\psi'(1) &= \kappa\end{aligned}$$

For a Poisson distribution, the excess degree equals the mean degree — a special property that simplifies the  $R_0$  expression.

## Polynomial PGF

For non-Poisson networks, the excess degree differs from the mean degree. Consider a network with degree distribution  $P(k=0) = 0.2$ ,  $P(k=1) = 0.3$ ,  $P(k=2) = 0.5$ :

```
pgf_poly = polynomial_pgf([0.2, 0.3, 0.5])
println("ψ(z) = ", pgf_poly.expression)
println("Mean degree: ", Symbolics.value(mean_degree(pgf_poly)))
```

$\psi(z) = 0.2 + 0.3z + 0.5(z^2)$   
Mean degree: 1.3

```
d1_poly = pgf_derivative(pgf_poly, 1)
d2_poly = pgf_derivative(pgf_poly, 2)

d1_at_1 = Symbolics.value(Symbolics.substitute(d1_poly, Dict{pgf_poly.variable => 1}))
d2_at_1 = Symbolics.value(Symbolics.substitute(d2_poly, Dict{pgf_poly.variable => 1}))

println("ψ'(1) = ", d1_at_1, " (mean degree)")
println("ψ''(1) = ", d2_at_1)
println("Excess degree ratio ψ''(1)/ψ'(1) = ", d2_at_1 / d1_at_1)
```

$\psi'(1) = 1.3$  (mean degree)  
 $\psi''(1) = 1$   
Excess degree ratio  $\psi''(1)/\psi'(1) = 0.7692307692307692$

For this distribution, the mean degree is 1.3 but the excess degree ratio is  $1.0/1.3 \approx 0.77$  — neighbours tend to have *fewer* additional contacts than the mean degree suggests. This is because the degree distribution is concentrated on low values ( $k \leq 2$ ).

## Comparing $R_0$ across degree distributions

We can use the symbolic framework to compare  $R_0$  for different network structures with the same mean degree:

```
@parameters β_plot γ_plot

# Poisson  $R_0$  for varying mean degree
κ_vals = 1.0:0.1:8.0
R0_poisson = [
```

```

    Symbolics.value(Symbolics.substitute(R0_sir, Dict( $\beta \Rightarrow 0.3$ ,  $\gamma \Rightarrow 0.1$ ,  $\kappa \Rightarrow k$ )))
    for k in  $\kappa$ _vals
]

# Polynomial PGF  $R_0$ : for each mean degree, construct a degree distribution
# with  $P(0) = 1 - \mu/k_{\max}$ , ..., concentrated on  $\{0, k_{\max}\}$ 
# Using a simple two-point distribution:  $P(0) = 1 - \mu/k_{\max}$ ,  $P(k_{\max}) = \mu/k_{\max}$ 
 $k_{\max} = 10$ 
R0_poly = Float64[]
for  $\mu$  in  $\kappa$ _vals
    if  $\mu > k_{\max}$ 
        push!(R0_poly, NaN)
        continue
    end
    probs = zeros( $k_{\max} + 1$ )
    probs[1] =  $1.0 - \mu / k_{\max}$            #  $P(k=0)$ 
    probs[ $k_{\max} + 1$ ] =  $\mu / k_{\max}$       #  $P(k=k_{\max})$ 
    pgf_p = polynomial_pgf(probs)
    prog_p = DiseaseProgression(
        [DiseaseStage(:I; transmission_rate= $\beta$ _plot), DiseaseStage(:R)],
        [DiseaseTransition(:I, :R,  $\gamma$ _plot)]; entry=:I
    )
    R0_p = basic_reproduction_number(StaticConfigurationModel(pgf_p, prog_p))
    push!(R0_poly, Symbolics.value(Symbolics.substitute(R0_p, Dict( $\beta$ _plot  $\Rightarrow 0.3$ ,  $\gamma$ _plot =
end

plot( $\kappa$ _vals, R0_poisson, lw=2, label="Poisson", color=:steelblue)
plot!( $\kappa$ _vals, R0_poly, lw=2, label="Two-point (0,  $k_{\max}$ )", color=:darkorange)
hline!([1.0], linestyle=:dash, color=:red, label="Epidemic threshold")
xlabel!("Mean degree")
ylabel!(" $R_0$ ")
title!(" $R_0$  by Degree Distribution")

```

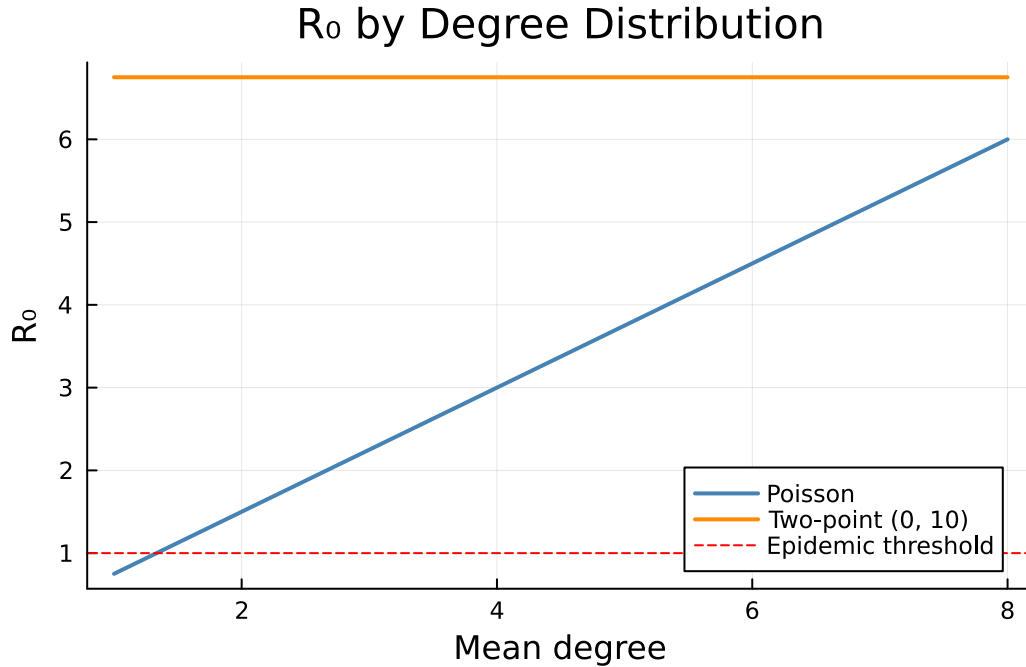


Figure 4:  $R_0$  for Poisson and polynomial degree distributions, both with the same mean degree. The Poisson network always has higher  $R_0$  because its excess degree equals its mean degree.

## Summary

The symbolic nature of EdgeBasedModels.jl enables powerful analytical workflows:

- $R_0$  computation: Exact symbolic expressions for the basic reproduction number, valid for arbitrary disease progressions and degree distributions.
- Equation inspection: Full visibility into the ODE systems before solving, useful for verification and teaching.
- Parameter sensitivity: Fast evaluation of  $R_0$  across parameter ranges without running ODE solvers.
- PGF analysis: Symbolic derivatives of the degree distribution's PGF reveal how network structure affects epidemic thresholds.

These capabilities complement the numerical simulation workflow shown in earlier vignettes, providing analytical insight that guides parameter choices and model validation.